

Guia de Rastreabilidade para Desenvolvedores

1. O que é Rastreabilidade

A rastreabilidade de requisitos é uma prática essencial em projetos de software que garante que cada requisito identificado (funcional ou não funcional) esteja diretamente vinculado a uma ou mais tarefas do time de desenvolvimento.

Essa prática permite acompanhar a implementação, evolução e validação de cada funcionalidade ao longo de todo o ciclo de vida do projeto.

2. Objetivo

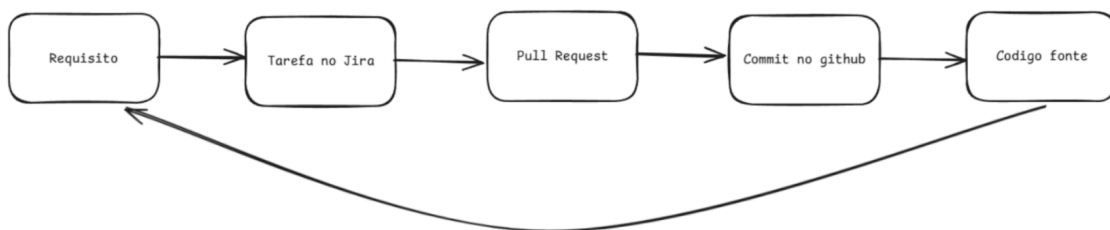
O principal objetivo da rastreabilidade é assegurar que todas as entregas atendam aos requisitos definidos, mantendo consistência, transparência e controle durante o desenvolvimento.

Com a rastreabilidade, é possível responder perguntas como:

- Este requisito já foi implementado?
- Qual tarefa ou código está vinculado a este requisito?
- Este commit atende a qual necessidade do sistema?

3. Processo de Rastreabilidade

- Cada requisito (RF ou RNF) deve estar descrito e numerado de forma única.
- Cada tarefa no Jira deve estar vinculada diretamente a um requisito.
- Cada Pull Request deve referenciar a tarefa do Jira correspondente.
- O código deve ser rastreável até o requisito que motivou sua implementação.



4. Instruções para Desenvolvedores

4.1 Nomenclatura de Branch (Obrigatória)

Padrão:

`tipo/TASK-REQUISITO-descricao`

Tipos permitidos:

`feature/` - Nova funcionalidade

`fix/` - Correção de bug

`hotfix/` - Correção urgente

`arch/` - Mudança arquitetural (impacto global)

`global/` - Alteração de estado/variável global (impacto global)

`config/` - Configuração que afeta todo o projeto

`maint/` - Manutenção ou melhoria

Exemplos:

`feature/API-31-RF-02-obter-dataset-funai`

`fix/API-45-RF-03-corrige-visualizacao-mapa`

`arch/API-50-RNF-05-implementa-microservicos`

`global/API-33-RF-05-estado-nlp-compartilhado`

4.2 Criação de Branch

- Sempre criar a branch diretamente pelo Jira (integração obrigatória).

- A branch deve herdar automaticamente o ID da task e o código do requisito.

- Não criar branches manualmente fora do padrão.

4.3 Padrão de Commits

Utilizar Conventional Commits simples, sem incluir códigos de Jira ou requisitos.

Tipos de commit:

`feat:` Nova funcionalidade

`fix:` Correção de bug

`docs:` Documentação

`style:` Formatação/estilo

`refactor:` Refatoração

`perf:` Performance

`test:` Testes

`build:` Build/dependências

`ci:` CI

`chore:` Manutenção

`revert:` Reversão

Exemplos:

feat: adiciona endpoint para importar dataset FUNAI
fix: corrige erro de timeout na conexão PostGIS
docs: atualiza documentação da API
refactor: reorganiza camadas
test: adiciona testes
perf: otimiza consultas
chore: atualiza dependências

4.4 Pull Requests — Regras Obrigatórias

Fluxo de branches:

feature -> dev (squash)
dev -> homolog (rebase)
homolog -> main (rebase)

Regras por branch:

dev: Squash and merge
homolog: Rebase and merge
main: Rebase and merge

Passo a passo:

1. Criar PR
2. Nome igual à branch
3. Code review
4. Merge correto

4.5 Code Review

- Todo PR deve passar por revisão.
- Verificar nomenclatura.
- Garantir rastreabilidade.

5. Situações Especiais

fix/API-67-RF-02-corrige-timeout-dataset
improvement/API-68-RF-02-otimiza-performance-dataset
maint/API-69-manutencao-RF-02-dataset

6. Rastreabilidade de Problemas em Produção

Fluxo:

1. Identificar commit
2. Localizar PR
3. Analisar branch
4. Ver Jira
5. Ver requisito
6. Buscar no repositório

```
git log --oneline --grep='palavra-chave'
```

7. Mudanças Críticas

arch/, global/, config/

Labels: impacto-global, mudanca-arquitetural, configuracao-critica

8. Documentação e Referências

<https://www.notion.so/Visiona-Documenta-o-349845ef97d9804cb0b5e29718f2e448>

9. Checklist do Desenvolvedor

Antes de criar branch:

- ☐ Task no Jira?
- ☐ Requisito claro?
- ☐ Nome correto?
- ☐ Mudanças críticas comunicadas?

Antes do PR:

- ☐ Commits padrão?
- ☐ Nome correto?
- ☐ Branch correta?

Antes do merge:

- ☐ PR aprovado?
- ☐ Método correto?
- ☐ Commit correto?